

Использование интегрального преобразования в PINN

Боева Галина, группа М05-304а

25 октября 2024 г.

1 Введение

Во многих вычислительных задачах в технике и науке дифференцирование функций или моделей имеет важное значение, но также необходимо и интегрирование. Важный класс вычислительных задач включает в себя так называемые интегро-дифференциальные уравнения, которые включают как интегралы, так и производные функции. В другом примере стохастические дифференциальные уравнения могут быть записаны в виде уравнения в частных производных для функции плотности вероятности стохастической переменной. Чтобы узнать характеристики для стохастической переменной, основанной на функции плотности, необходимо вычислить конкретные интегральные преобразования, а именно моменты, функции плотности. В последнее время парадигма машинного обучения в нейронных сетях, основанных на физике, приобретают все большую популярность как метод решения дифференциальных уравнений с использованием автоматического дифференцирования. В этой работе мы предлагаем дополнить парадигму нейронных сетей, основанных на физике, автоматической интеграцией, чтобы вычислять сложные интегральные преобразования для обученных решений и решать интегро-дифференциальные уравнения.

2 Определение PINN

Физически информированное машинное обучение предлагает решать дифференциальные уравнения, представляя приближенное решение с помощью искусственной нейронной сети, дифференцируемой квантовой схемы (DQC) или другого универсального функционального аппроксиматора (UFA) и оптимизируя UFA относительно функции потерь, построенной на основе дифференциальных уравнений. Параметризованное дифференциальное уравнение формально записывается как

$$P(f, x; \lambda) = 0,$$

где P — нелинейный оператор, действующий на $f(x)$, а λ — параметр (обратите внимание, что x и λ могут быть многомерными). Если решение, аппроксимированное UFA, представлено как u_θ , то сходимость u_θ к решению дифференциального

уравнения (в большинстве случаев) обеспечивается условием

$$\|P(u_\theta, x; \lambda)\| = 0.$$

Функция потерь для решения дифференциальных уравнений в физически информированном машинном обучении вдохновлена вышеупомянутым наблюдением и определяется как

$$\sum_{x_i} \|P(u_\theta, x_i; \lambda)\| = 0,$$

где x_i представляет конечное число точек, выбранных из области определения дифференциального уравнения. UFA, такие как искусственные нейронные сети и дифференцируемые квантовые схемы (DQC), облегчают вычисление этой функции потерь с помощью автоматического дифференцирования.

3 Автоматическая интегрирование

Рассмотрим нейронную сеть

$$f_\theta : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R} \quad (s, v) \mapsto f_\theta(s, v).$$

По основной теореме исчисления, частная антипроизводная F_θ от f_θ удовлетворяет

$$f_\theta(s, v) = \int_{s_0}^s \frac{\partial f_\theta(q, v)}{\partial q} dq = \int_{s_0}^s F_\theta(s_0, v) ds_0.$$

Обратите внимание, что $F_\theta(s, v)$ также является функцией с той же областью и кодоменом, что и $f_\theta(s, v)$, и может быть представлена нейронной сетью, которая использует те же параметры θ , что и f_θ . Lindell et al. отмечают, что можно оптимизировать F_θ для представления некоторых целевых выходных данных и получить f_θ без дополнительной оптимизации из оптимизированных параметров F_θ .

Мы предлагаем дополнить парадигму PINN автоматическим интегрированием для вычисления интеграла функций, обученных с помощью PINN; кроме того, мы расширяем применение от интеграции функций до более общего понятия интегральных преобразований функций.

4 Интегральные преобразования

Интегральное преобразование — это математический оператор, который отображает функцию f в другую функцию $\hat{f}$, определенную как

$$\hat{f}(x, y) = \int k(x, x_0) f(x_0, y) dx_0,$$

где функция $k(x, x_0)$ известна как ядро интегрального преобразования.

Интегральное преобразование возникает в широком спектре областей. Оно часто используется для преобразования задачи, которая слишком сложна для решения в своем исходном представлении, например, для преобразования дифференциального уравнения в алгебраическое уравнение с помощью преобразования

Фурье. Оно также часто возникает в областях, использующих вероятность или статистику, например, для гауссового сглаживания данных.

Если ядро является тождественным, интегральное преобразование сводится к обычному интегралу. Существуют различные стандартные преобразования, такие как преобразование Фурье и преобразование Лапласа, ядро которых фиксировано. Ядро фундаментальных решений дифференциальных уравнений и функций Грина, с другой стороны, может зависеть от дифференциальных уравнений и их граничных условий.

5 Интегральные преобразования для функций, обученных с помощью PINN

Алгоритм
1. Вход Комбинация: (интегро-)дифференциальных уравнений, граничных условий, данных. 2. Преобразование Преобразуйте весь вход с помощью: $f(s, v) \rightarrow \frac{1}{k(s, s_0)} \frac{\partial G(q, s_0, v)}{\partial q} \Big _{q=s}$ $\int_{s_b}^{s_a} k(s, s_0) f(s, v) ds \rightarrow G(s_b, s_0, v) - G(s_a, s_0, v).$ 3. Обучение Обучите G с использованием дифференцируемой модели (PINN, DQC и т.д.). 4. Выход Используйте обученную G для вычисления интегрального преобразования или неизвестной функции: $\int_{s_b}^{s_a} k(s, s_0) f(s, v) ds = G(s_b, s_0, v) - G(s_a, s_0, v)$ $f(s, v) = \frac{1}{k(s, s_0)} \frac{\partial G(q, s_0, v)}{\partial q} \Big _{q=s}.$

Таблица 1: Алгоритм.

В этой работе мы объединяем физически информированные (квантовые) нейронные сети и автоматическое интегрирование для формулировки квантово-совместимого метода машинного обучения для выполнения общего интегрального преобразования на обученной функции. Рассмотрим неизвестную функцию

$$f : \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R} \quad (s, v) \mapsto f(s, v).$$

Некоторая информация о функции f известна в виде:

- значения f или ее производных в определенных частях области.
- интегро-дифференциальных уравнений неизвестной функции f в переменных s и v .

Мы обсуждаем метод для представления в настройке PINN интегрального преобразования

$$\hat{f}(s_0, v) = \int_{s_a(s_0)}^{s_b(s_0)} k(s, s_0) f(s, v) ds,$$

где функции ядра $k(s, s_0) \in \mathbb{R}$, $s_a(s_0) \in \mathbb{R}$ и $s_b(s_0) \in \mathbb{R}$ известны. Этот метод затем используется для:

- оценки интегрального преобразования функции без явного обучения самой функции.
- решения интегро-дифференциальных уравнений для неизвестной функции f .

Сначала мы определяем вспомогательную функцию

$$G : \mathbb{R} \times \mathbb{R} \times \mathbb{R}^n \rightarrow \mathbb{R} \quad (s, s_0, v) \mapsto G(s, s_0, v).$$

Если s_a и s_b являются функциями s_0 , мы определяем $k(s, s_0) = 0$, если $s < s_a(s_0)$ или $s > s_b(s_0)$.

Следующим шагом является переформулировка данных и (интегро-)дифференциальных уравнений, заданных в терминах f , в терминах вспомогательной функции G . Это делается с помощью подстановок

$$f(s, v) \rightarrow \frac{1}{k(s, s_0)} \frac{\partial G(q, s_0, v)}{\partial q} \Big|_{q=s}$$

$$\int_{s_b}^{s_a} k(s, s_0) f(s, v) ds \rightarrow G(s_b, s_0, v) - G(s_a, s_0, v).$$

С этими подстановками вся задача теперь сформулирована в терминах G и ее производных, без каких-либо ссылок на f , ее производные или ее интеграл. Универсальный функциональный аппроксиматор (UFA) в настройке PINN, такой как классическая нейронная сеть или DQC, теперь может быть использован для прямого обучения G . После завершения обучения интегральное преобразование неизвестной функции или сама неизвестная функция могут быть вычислены из уравнения (12). Сводка алгоритма показана в Таблице I.

6 Интегральные преобразования на примере

6.1 Преобразование Фурье

Преобразование Фурье функции $f(x)$ определяется как:

$$\mathcal{F}\{f(x)\} = \hat{f}(k) = \int_{-\infty}^{\infty} f(x) e^{-ikx} dx$$

Обратное преобразование Фурье:

$$\mathcal{F}^{-1}\{\hat{f}(k)\} = f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(k) e^{ikx} dk$$

6.2 Пример: Уравнение диффузии

Рассмотрим уравнение диффузии:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}$$

Применим преобразование Фурье по переменной x :

$$\begin{aligned}\mathcal{F} \left\{ \frac{\partial u}{\partial t} \right\} &= \frac{\partial \hat{u}(k, t)}{\partial t} \\ \mathcal{F} \left\{ \nu \frac{\partial^2 u}{\partial x^2} \right\} &= -\nu k^2 \hat{u}(k, t)\end{aligned}$$

Таким образом, уравнение в частотном пространстве становится:

$$\frac{\partial \hat{u}(k, t)}{\partial t} = -\nu k^2 \hat{u}(k, t)$$

Это уравнение можно решить аналитически:

$$\hat{u}(k, t) = \hat{u}(k, 0) e^{-\nu k^2 t}$$

где $\hat{u}(k, 0)$ — начальное условие в частотном пространстве.

7 PINN и интегральные преобразования

Предположим, у нас есть нейронная сеть $\hat{u}(x, t; \theta)$, которая аппроксимирует $u(x, t)$, где θ — параметры сети.

Функция потерь L включает два компонента:

1. Потеря на данных:

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N (\hat{u}(x_i, t_i; \theta) - u_i)^2$$

2. Потеря на физическом уравнении:

$$L_{\text{physics}} = \frac{1}{M} \sum_{j=1}^M \left(\frac{\partial \hat{u}(k_j, t_j; \theta)}{\partial t} + \nu k_j^2 \hat{u}(k_j, t_j; \theta) \right)^2$$

Общая функция потерь:

$$L = L_{\text{data}} + \lambda L_{\text{physics}}$$

где λ — гиперпараметр, балансирующий влияние данных и физических законов.